

UNIT 1

Introduction & Foundational Neural Networks

Roadmap of This Unit

The unit is organized into six blocks:

- Introduction to Deep Learning — AI vs ML vs DL, History and Milestones
- Biological & Artificial Neurons — Biological Neuron, McCulloch-Pitts Model
- Perceptron Learning — Perceptron Learning Rule, Perceptron Convergence Theorem
- Feedforward Networks — Multilayer Perceptrons (MLPs), Deep Feedforward Networks
- Activation Functions — Sigmoid, Tanh, ReLU, Leaky ReLU, ELU
- Loss Functions & Backpropagation — MSE, Cross-Entropy, Binary Cross-Entropy, Forward Propagation, Backpropagation

1. Introduction to Deep Learning

1.1 AI vs ML vs DL

Artificial Intelligence (AI), Machine Learning (ML), and Deep Learning (DL) are nested fields, not synonyms.

AI is the broadest goal: building systems that exhibit intelligent behaviour, whether through hand-coded rules, search algorithms, or learned models.

Machine Learning is a subset of AI in which a system improves at a task by learning patterns from data rather than following explicitly programmed rules.

Deep Learning is, in turn, a subset of Machine Learning that uses neural networks with many layers ('depth') to automatically learn hierarchical representations of data, removing much of the manual feature-engineering that classical ML requires.

Field	Core Idea	Example
Artificial Intelligence	Any technique that makes machines act intelligently	Rule-based chess engine, expert systems
Machine Learning	Learn patterns/rules from data instead of hand-coding them	Decision trees, SVMs, linear regression
Deep Learning	Learn layered/hierarchical representations via neural networks	CNNs for image recognition, Transformers for language

A useful way to see the distinction: a classical ML pipeline for image classification typically needs a human to hand-design features (edges, corners, color histograms) which are then fed to a learning algorithm.

A deep learning pipeline instead feeds raw pixels directly into a deep network, which learns the useful features itself, layer by layer — early layers learning edges, middle layers learning shapes, and later layers learning object parts.

1.2 History and Milestones of Deep Learning

Year	Milestone
1943	McCulloch & Pitts propose the first mathematical model of an artificial neuron
1958	Rosenblatt introduces the Perceptron and its learning rule
1969	Minsky & Papert publish 'Perceptrons', showing single-layer perceptrons cannot solve XOR — this contributed to the first 'AI winter'
1986	Rumelhart, Hinton & Williams popularize backpropagation, enabling multilayer network training
1998	LeCun's LeNet-5 demonstrates convolutional neural networks for digit recognition
2006	Hinton popularizes 'deep learning' via layer-wise pretraining of deep belief nets
2012	AlexNet wins ImageNet by a huge margin, igniting the modern deep learning boom
2014	Generative Adversarial Networks (GANs) introduced by Goodfellow et al.
2017	The Transformer architecture ('Attention Is All You Need') is introduced
2018–present	Large pretrained models (BERT, GPT family, diffusion models) drive rapid progress across language, vision, and multimodal tasks

2. Biological & Artificial Neurons

2.1 The Biological Neuron

A biological neuron receives electrochemical signals through branch-like structures called dendrites, integrates those signals in the cell body (soma), and — if the combined signal exceeds a threshold — fires an electrical pulse (an action potential) down its axon. That pulse is passed to other neurons through synapses, junctions whose 'strength' can increase or decrease with experience. This threshold-based firing and adjustable connection strength is the direct biological inspiration for artificial neurons: weighted inputs, a summation step, an activation/threshold function, and learnable weights that play the role of synaptic strength.

Biological Component	Artificial Neuron Analogue
Dendrites	Input connections ($x_1, x_2, \dots, x_n$)
Synaptic strength	Weights ($w_1, w_2, \dots, w_n$)
Cell body (soma) summation	Weighted sum: $z = \sum w_i x_i + b$
Firing threshold	Activation function (e.g. step, sigmoid, ReLU)
Axon output	Neuron's output value, passed to the next layer

2.2 The McCulloch-Pitts (M-P) Model

Proposed in 1943, the McCulloch-Pitts neuron is the earliest mathematical model of a neuron. It takes binary inputs, sums them (optionally with binary inhibitory inputs that force the output to 0 if active), and fires (outputs 1) only if the sum meets or exceeds a fixed threshold θ . It has no learnable weights — the threshold and connections are hand-set, not learned — but it proved that networks of simple threshold units could, in principle, compute any logical function, laying the theoretical groundwork for everything that followed.

$$y = 1 \text{ if } \sum x_i \geq \theta \text{ and no inhibitory input is active } y = 0 \text{ otherwise}$$

3. Perceptron Learning

Rosenblatt's Perceptron (1958) extended the McCulloch-Pitts neuron with learnable, real-valued weights and a training procedure. Rather than hand-setting a threshold, the perceptron starts with (typically) small or zero weights and adjusts them automatically based on its mistakes, making it the first true learning algorithm for an artificial neuron.

3.1 Perceptron Architecture

A perceptron computes a weighted sum of its inputs plus a bias, then passes that sum through a step (Heaviside) activation function to produce a binary output.

$$z = \sum_i w_i x_i + b$$
$$\hat{y} = 1 \text{ if } z \geq 0, \text{ else } 0$$

3.2 The Perceptron Learning Rule

For each training example, the perceptron makes a prediction and compares it to the true label. If the prediction is correct, nothing changes. If it is wrong, every weight is nudged in the direction that would have made the correct answer more likely, scaled by a learning rate η .

$$w_i := w_i + \eta \cdot (y - \hat{y}) \cdot x_i$$
$$b := b + \eta \cdot (y - \hat{y})$$

Because $(y - \hat{y})$ is 0 when the prediction is correct, +1 when the true label is 1 but the prediction was 0, and -1 when the true label is 0 but the prediction was 1, this rule only fires — and only moves the decision boundary — on misclassified examples.

Practical Implementation — Perceptron Learning Rule

```
import numpy as np

class Perceptron:
    def __init__(self, n_features, lr=0.1):
        self.w = np.zeros(n_features)
        self.b = 0.0
        self.lr = lr

    def step_function(self, z):
        return np.where(z >= 0, 1, 0)

    def predict(self, X):
        z = X @ self.w + self.b
        return self.step_function(z)

    def fit(self, X, y, epochs=20):
```

```

history = []
for epoch in range(epochs):
    errors = 0
    for xi, yi in zip(X, y):
        z = np.dot(xi, self.w) + self.b
        y_pred = 1 if z >= 0 else 0
        update = self.lr * (yi - y_pred)

        if update != 0:
            self.w += update * xi
            self.b += update
            errors += 1
    history.append(errors)
    if errors == 0:
        print(f"Converged after {epoch + 1} epochs.")
        break
return history

# Train an AND-gate perceptron
X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([0, 0, 0, 1]) # AND

model = Perceptron(n_features=2, lr=0.1)
history = model.fit(X, y, epochs=20)

print("Learned weights:", model.w, "bias:", model.b)
print("Predictions:", model.predict(X))

```

3.3 The Perceptron Convergence Theorem

The Perceptron Convergence Theorem guarantees that if the training data is linearly separable — meaning some hyperplane exists that perfectly separates the two classes — the perceptron learning rule is guaranteed to find a separating hyperplane in a finite number of updates, regardless of the initial weights. If the data is NOT linearly separable (such as XOR), the algorithm will never converge and will keep oscillating indefinitely.

Practical Demonstration — Convergence vs Non-Convergence

```

# Case 1: Linearly separable data (AND gate) -> perceptron converges
X_and = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_and = np.array([0, 0, 0, 1])

model_and = Perceptron(n_features=2, lr=0.1)
errors_and = model_and.fit(X_and, y_and, epochs=50)
print("AND gate error history:", errors_and)
print("-> Reaches 0 errors: linearly separable, converges.\n")

# Case 2: NOT linearly separable (XOR) -> perceptron never converges
X_xor = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y_xor = np.array([0, 1, 1, 0])

```

```
model_xor = Perceptron(n_features=2, lr=0.1)
errors_xor = model_xor.fit(X_xor, y_xor, epochs=50)
print("XOR error history (first 10 epochs):", errors_xor[:10])
print("-> Error count never reaches 0: not linearly separable, does not converge.")
```

Key takeaway: XOR is the classic counter-example that motivated moving beyond a single perceptron to multilayer networks — see Section 4.

4. Feedforward Networks

4.1 Why a Single Perceptron Isn't Enough

A single perceptron computes a linear decision boundary — geometrically, a straight line (in 2D) or hyperplane (in higher dimensions). XOR's positive and negative examples cannot be separated by any single straight line, so no single perceptron, however well trained, can represent it. The fix is architectural: stack multiple layers of neurons so the network as a whole can compose several linear boundaries into a non-linear one.

4.2 Multilayer Perceptrons (MLPs)

A Multilayer Perceptron consists of an input layer, one or more hidden layers, and an output layer, where every neuron in one layer connects to every neuron in the next (a 'fully connected' or 'dense' architecture). Each hidden neuron computes a weighted sum of its inputs followed by a non-linear activation function (Section 5); it is precisely this non-linearity between layers that lets an MLP carve out non-linear decision boundaries like the one XOR requires.

$$\begin{aligned}a^{(1)} &= f(W^{(1)}x + b^{(1)}) \\ a^{(2)} &= f(W^{(2)}a^{(1)} + b^{(2)}) \\ \hat{y} &= g(W^{(L)}a^{(L-1)} + b^{(L)})\end{aligned}$$

Here f is a hidden-layer activation function (e.g. ReLU) and g is an output activation suited to the task (e.g. sigmoid for binary classification, softmax for multi-class).

Practical Implementation — 2-Layer MLP Solving XOR

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def sigmoid_derivative(a):
    return a * (1 - a)  # a is already sigmoid(z)

class TwoLayerMLP:
    def __init__(self, n_input, n_hidden, n_output, lr=0.5, seed=1):
        rng = np.random.default_rng(seed)
        self.W1 = rng.normal(0, 1, (n_input, n_hidden))
        self.b1 = np.zeros((1, n_hidden))
        self.W2 = rng.normal(0, 1, (n_hidden, n_output))
        self.b2 = np.zeros((1, n_output))
        self.lr = lr

    def forward(self, X):
        self.z1 = X @ self.W1 + self.b1
        self.a1 = sigmoid(self.z1)
        self.z2 = self.a1 @ self.W2 + self.b2
```

```

        self.a2 = sigmoid(self.z2)
        return self.a2

def backward(self, X, y):
    m = X.shape[0]

    # Output layer error
    dz2 = self.a2 - y
    dW2 = (self.a1.T @ dz2) / m
    db2 = np.sum(dz2, axis=0, keepdims=True) / m

    # Hidden layer error (propagate through W2)
    da1 = dz2 @ self.W2.T
    dz1 = da1 * sigmoid_derivative(self.a1)
    dW1 = (X.T @ dz1) / m
    db1 = np.sum(dz1, axis=0, keepdims=True) / m

    self.W2 -= self.lr * dW2
    self.b2 -= self.lr * db2
    self.W1 -= self.lr * dW1
    self.b1 -= self.lr * db1

def train(self, X, y, epochs=5000):
    losses = []
    for epoch in range(epochs):
        y_pred = self.forward(X)
        loss = np.mean((y - y_pred) ** 2)
        losses.append(loss)
        self.backward(X, y)
    return losses

X = np.array([[0, 0], [0, 1], [1, 0], [1, 1]])
y = np.array([[0], [1], [1], [0]]) # XOR

mlp = TwoLayerMLP(n_input=2, n_hidden=4, n_output=1, lr=0.5)
losses = mlp.train(X, y, epochs=5000)

print("Final loss:", round(losses[-1], 6))
print("Predictions:")
for xi, pred in zip(X, mlp.forward(X)):
    print(f" {xi} -> {pred[0]:.4f} (rounded: {round(pred[0])})")
# A single perceptron could never do this; the hidden layer makes it possible.

```

4.3 Deep Feedforward Networks

A Deep Feedforward Network (also called a Deep Neural Network, DNN) simply extends the MLP idea to many hidden layers stacked in sequence, with information flowing strictly forward from input to output (no cycles or feedback — which is what distinguishes 'feedforward' from recurrent architectures). Depth allows the network to build increasingly abstract, hierarchical representations: shallow layers detect simple patterns, and deeper layers combine those into complex, task-relevant features.

Practical Implementation — Generic N-Layer Feedforward Network

```
import numpy as np

def relu(z):
    return np.maximum(0, z)

def relu_derivative(z):
    return (z > 0).astype(float)

class DeepFeedforwardNet:
    def __init__(self, layer_sizes, lr=0.01, seed=0):
        """layer_sizes: e.g. [4, 8, 8, 1] -> input=4, two hidden layers of 8, output=1"""
        rng = np.random.default_rng(seed)
        self.lr = lr
        self.L = len(layer_sizes) - 1
        self.W = [rng.normal(0, np.sqrt(2 / layer_sizes[i]), (layer_sizes[i],
layer_sizes[i+1]))
                    for i in range(self.L)]
        self.b = [np.zeros((1, layer_sizes[i+1])) for i in range(self.L)]

    def forward(self, X):
        self.z = []
        self.a = [X]
        for i in range(self.L):
            z = self.a[-1] @ self.W[i] + self.b[i]
            self.z.append(z)
            # ReLU for hidden layers, linear (identity) for the final output layer
            a = relu(z) if i < self.L - 1 else z
            self.a.append(a)
        return self.a[-1]

    def backward(self, y):
        m = y.shape[0]
        grads_W, grads_b = [None] * self.L, [None] * self.L

        dz = (self.a[-1] - y) / m # MSE-derivative for the final linear layer
        for i in reversed(range(self.L)):
            grads_W[i] = self.a[i].T @ dz
            grads_b[i] = np.sum(dz, axis=0, keepdims=True)
            if i > 0:
                da_prev = dz @ self.W[i].T
                dz = da_prev * relu_derivative(self.z[i - 1])

        for i in range(self.L):
            self.W[i] -= self.lr * grads_W[i]
            self.b[i] -= self.lr * grads_b[i]

    def train(self, X, y, epochs=500):
        losses = []
        for _ in range(epochs):
            y_pred = self.forward(X)
            losses.append(np.mean((y_pred - y) ** 2))
            self.backward(y)
```

```
    return losses
```

```
# Example: a 3-hidden-layer deep feedforward network on synthetic regression data
```

```
np.random.seed(0)
```

```
X = np.random.randn(100, 4)
```

```
y = (X[:, [0]] * 2 - X[:, [1]] * 3 + X[:, [2]] ** 2).reshape(-1, 1)
```

```
net = DeepFeedforwardNet(layer_sizes=[4, 16, 16, 1], lr=0.01)
```

```
losses = net.train(X, y, epochs=500)
```

```
print(f"Loss: start={losses[0]:.4f} -> end={losses[-1]:.4f}")
```

5. Activation Functions

Activation functions introduce non-linearity between layers. Without them, stacking any number of layers would collapse mathematically into a single linear transformation (a composition of linear functions is still linear), so the network would be no more powerful than a single perceptron. The choice of activation function affects how gradients flow during backpropagation, so it has a direct effect on training speed and stability (recall Unit 2, Section 4 on vanishing/exploding gradients).

5.1 Sigmoid

The sigmoid function squashes any real input into the range (0, 1), making it a natural choice for representing probabilities. However, its derivative is small almost everywhere and saturates (flattens) for large positive or negative inputs, which makes it a major contributor to the vanishing gradient problem in deep networks.

$$\sigma(z) = 1 / (1 + e^{-z}) \quad \sigma'(z) = \sigma(z) \cdot (1 - \sigma(z))$$

5.2 Tanh (Hyperbolic Tangent)

Tanh is a rescaled sigmoid that outputs values in (-1, 1) and is zero-centered, which generally leads to faster convergence than sigmoid since its outputs don't push all downstream weight updates in the same direction. It still saturates for large $|z|$, so it also suffers from vanishing gradients, though less severely than sigmoid.

$$\tanh(z) = (e^z - e^{-z}) / (e^z + e^{-z}) \quad \tanh'(z) = 1 - \tanh^2(z)$$

5.3 ReLU (Rectified Linear Unit)

ReLU outputs the input directly if it is positive, and zero otherwise. Its derivative is exactly 1 for all positive inputs (no saturation, no shrinking gradient) and is very cheap to compute, which is why it is the default choice for hidden layers in most modern architectures. Its main drawback is the 'dying ReLU' problem: a neuron whose input is always negative outputs zero and has zero gradient, so it can get permanently stuck and stop learning.

$$\text{ReLU}(z) = \max(0, z) \quad \text{ReLU}'(z) = 1 \text{ if } z > 0, \text{ else } 0$$

5.4 Leaky ReLU

Leaky ReLU fixes the dying ReLU problem by allowing a small, non-zero gradient (controlled by a small constant α , e.g. 0.01) when the input is negative, instead of hard-clipping it to zero. This keeps a trickle of gradient flowing even for negative inputs, so neurons are less likely to become permanently inactive.

$$\text{LeakyReLU}(z) = z \text{ if } z > 0, \text{ else } \alpha \cdot z \quad \text{LeakyReLU}'(z) = 1 \text{ if } z > 0, \text{ else } \alpha$$

5.5 ELU (Exponential Linear Unit)

ELU behaves like ReLU for positive inputs but smoothly saturates to $-\alpha$ for negative inputs using an exponential curve, rather than a straight line. This keeps mean activations closer to zero (helping with faster convergence, similar to the benefit tanh has over sigmoid) while still avoiding the dying-unit problem, at the cost of being more expensive to compute than ReLU/Leaky ReLU due to the exponential.

$$ELU(z) = z \text{ if } z > 0, \text{ else } \alpha \cdot (e^z - 1) \quad ELU'(z) = 1 \text{ if } z > 0, \text{ else } ELU(z) + \alpha$$

Practical Implementation — All Five Activation Functions

```
import numpy as np

def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def sigmoid_derivative(z):
    s = sigmoid(z)
    return s * (1 - s)

def tanh(z):
    return np.tanh(z)

def tanh_derivative(z):
    return 1 - np.tanh(z) ** 2

def relu(z):
    return np.maximum(0, z)

def relu_derivative(z):
    return (z > 0).astype(float)

def leaky_relu(z, alpha=0.01):
    return np.where(z > 0, z, alpha * z)

def leaky_relu_derivative(z, alpha=0.01):
    return np.where(z > 0, 1.0, alpha)

def elu(z, alpha=1.0):
    return np.where(z > 0, z, alpha * (np.exp(z) - 1))

def elu_derivative(z, alpha=1.0):
    return np.where(z > 0, 1.0, elu(z, alpha) + alpha)

# Compare all activations on the same input range
z = np.array([-3, -1, -0.1, 0, 0.1, 1, 3])

print(f"{'z':>6} {'Sigmoid':>9} {'Tanh':>8} {'ReLU':>7} {'LReLU':>8} {'ELU':>8}")
for zi in z:
    print(f"{zi:6.1f} {sigmoid(zi):9.4f} {tanh(zi):8.4f} {relu(zi):7.4f} "
          f"{leaky_relu(zi):8.4f} {elu(zi):8.4f}")
```

```

print("\nGradient at z = -3 (illustrates vanishing/dying-unit behavior):")
print(f"  Sigmoid':    {sigmoid_derivative(-3):.6f}  (nearly vanished)")
print(f"  Tanh':        {tanh_derivative(-3):.6f}  (nearly vanished)")
print(f"  ReLU':         {relu_derivative(-3):.6f}  (exactly zero -> 'dead' neuron)")
print(f"  LeakyReLU':    {leaky_relu_derivative(-3):.6f}  (small but alive)")
print(f"  ELU':          {elu_derivative(-3):.6f}  (small but alive, smooth)")

```

Function	Output Range	Zero-centered?	Saturates?	Typical Use
Sigmoid	$(0, 1)$	No	Yes (both sides)	Output layer for binary classification
Tanh	$(-1, 1)$	Yes	Yes (both sides)	Hidden layers in older/shallow nets, RNNs
ReLU	$[0, \infty)$	No	No (positive side)	Default for hidden layers in most modern nets
Leaky ReLU	$(-\infty, \infty)$	Approx.	No	Hidden layers where dying ReLU is a concern
ELU	$(-\alpha, \infty)$	Approx.	No (smooth negative)	Hidden layers wanting smoother negative response

6. Loss Functions & Backpropagation

A loss function measures how far the network's prediction is from the true target — training is the process of adjusting weights to minimize this value. Backpropagation is the algorithm that efficiently computes how the loss changes with respect to every weight in the network (its gradient), by applying the chain rule backward from the output layer to the input layer.

6.1 Mean Squared Error (MSE)

MSE averages the squared difference between predicted and true values, and is the standard loss for regression tasks. Squaring penalizes large errors disproportionately more than small ones and keeps the loss differentiable everywhere.

$$MSE = (1/N) \sum_i (y_i - \hat{y}_i)^2 \quad \partial MSE / \partial \hat{y}_i = -(2/N)(y_i - \hat{y}_i)$$

6.2 Cross-Entropy Loss (Multi-class)

Cross-Entropy measures the divergence between the true class distribution (typically a one-hot vector) and the predicted probability distribution (usually the output of a softmax layer). It is the standard loss for multi-class classification and grows sharply as the predicted probability for the true class approaches zero, strongly penalizing confident-but-wrong predictions.

$$CE = -\sum_i y_i \cdot \log(\hat{y}_i)$$

6.3 Binary Cross-Entropy (BCE)

Binary Cross-Entropy is the two-class special case of cross-entropy, used with a single sigmoid output for binary classification. It compares the true binary label to the predicted probability of the positive class.

$$BCE = -(1/N) \sum_i [y_i \cdot \log(\hat{y}_i) + (1 - y_i) \cdot \log(1 - \hat{y}_i)]$$

Practical Implementation — MSE, Cross-Entropy, and BCE

```
import numpy as np

def mse(y_true, y_pred):
    return np.mean((y_true - y_pred) ** 2)

def mse_derivative(y_true, y_pred):
    return -2 * (y_true - y_pred) / y_true.size

def cross_entropy(y_true, y_pred, eps=1e-12):
    y_pred = np.clip(y_pred, eps, 1 - eps) # avoid log(0)
    return -np.sum(y_true * np.log(y_pred)) / y_true.shape[0]

def binary_cross_entropy(y_true, y_pred, eps=1e-12):
    y_pred = np.clip(y_pred, eps, 1 - eps)
    return -np.mean(y_true * np.log(y_pred) + (1 - y_true) * np.log(1 - y_pred))

# MSE example (regression)
y_true_reg = np.array([3.0, 5.0, 2.5])
y_pred_reg = np.array([2.8, 5.2, 2.0])
print("MSE:", round(mse(y_true_reg, y_pred_reg), 4))

# Cross-Entropy example (3-class, one-hot true label)
y_true_ce = np.array([[0, 1, 0]])
y_pred_ce = np.array([[0.1, 0.7, 0.2]]) # softmax output
print("Cross-Entropy:", round(cross_entropy(y_true_ce, y_pred_ce), 4))

# Binary Cross-Entropy example
y_true_bce = np.array([1, 0, 1, 1])
y_pred_bce = np.array([0.9, 0.2, 0.6, 0.8]) # sigmoid outputs
print("Binary Cross-Entropy:", round(binary_cross_entropy(y_true_bce, y_pred_bce), 4))
```

6.4 Forward Propagation

Forward propagation is the process of passing an input through the network, layer by layer, to produce a prediction. At each layer, the previous layer's activations are linearly transformed (multiplied by weights, plus a bias) and passed through a non-linear activation function; the final layer's output is the network's prediction, which is then compared to the true label using a loss function.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}$$
$$a^{(l)} = f(z^{(l)})$$

6.5 Backpropagation

Backpropagation computes the gradient of the loss with respect to every weight and bias by applying the chain rule from the output layer backward to the input layer, reusing intermediate results (the 'error' signal δ at each layer) so that the entire gradient computation costs roughly the same as one extra forward pass — this efficiency is exactly what makes training deep networks computationally feasible.